

DOCKER – TASK – 02

1. CREATE A TOMCAT CONTAINER ON 8080 AND DEPLOY SAMPLE APPLICATION IN TOMCAT.

Start Docker Service

- systemctl start docker
- systemctl status docker
- docker images
Shows all images already downloaded on the server.
- Download Tomcat Image
- docker pull tomcat
Downloads the latest Tomcat image from Docker Hub.

```
ubuntu@ip-172-31-4-171:~$ sudo su -
root@ip-172-31-4-171:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-25 14:12:50 UTC; 44s ago
     Invocation: ee451ab124ec4f81abb67b45a90849cf
   TriggeredBy: ● docker.socket
                Docs: https://docs.docker.com
   Main PID: 1035 (dockerd)
    Tasks: 11
   Memory: 93.5M (peak: 93.8M)
      CPU: 318ms
   CGroup: /system.slice/docker.service
           └─1035 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

docker: unknown command: docker iamges
```

```
Run 'docker --help' for more information
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	U
nginx:latest	0d4374c710a9	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

```
root@ip-172-31-4-171:~# docker pull tomcat
Using default tag: latest
latest: Pulling from library/tomcat
4f4fb700ef54: Pull complete
b385a54fa58d: Pull complete
6bd4a5f61a04: Pull complete
da76df643e43: Download complete
d477108507e1: Download complete
Digest: sha256:b4237a8551b327a88d67c156a943e2db14eb2bcbaa85cbfebd587e0cab9c8885
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	U
nginx:latest	0d4374c710a9	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
tomcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	

Run Tomcat Container

➤ `docker run -itd -p 8080:8080 tomcat`

What happened:

- `-i` → Interactive
- `-t` → Terminal
- `-d` → Run in background
- `-p 8080:8080` → Map server port to container port
- `tomcat` → Image name

```
root@ip-172-31-4-171:~# docker run -itd -p 8080:8080 tomcat
299ecd39c568c9dc905cf059c7ab6fd4fecea91997640c8354267284644311
root@ip-172-31-4-171:~#
```

Open Tomcat in Browser

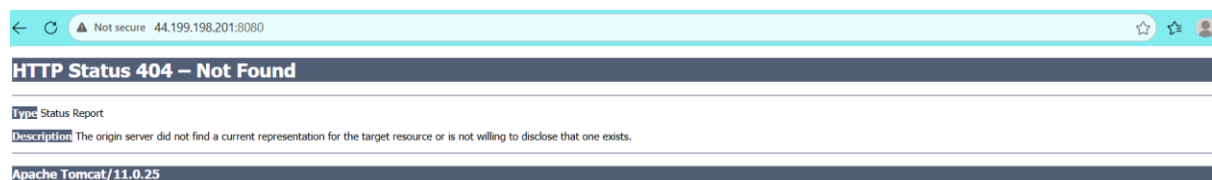
<http://44.199.198.201>

What happened:

- Browser showed 404 Not Found.

Why?

- Default Tomcat page is disabled.
- No application was deployed yet.



Check Running Containers

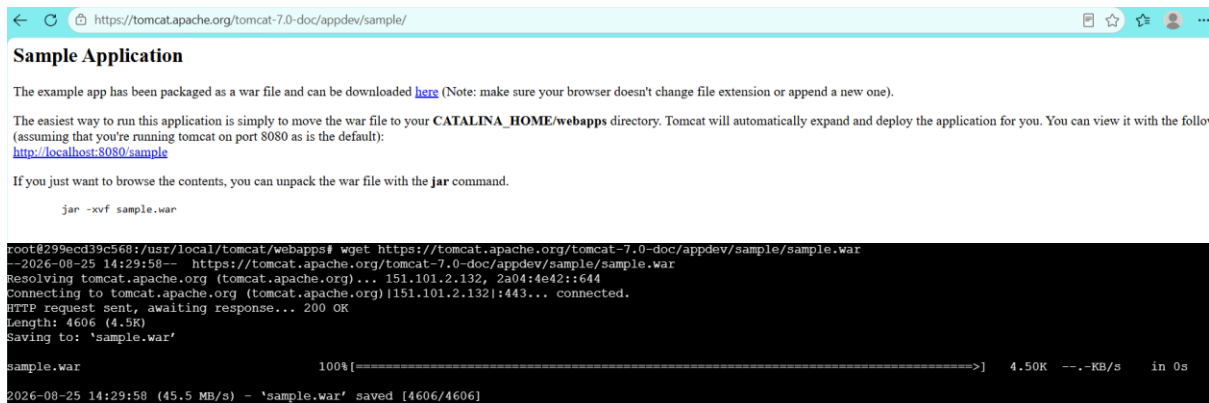
➤ `docker ps -a`

What happened:

- Shows all containers (running + stopped).

```
root@ip-172-31-4-171:~# docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS      PORTS                               NAMES
299ecd39c568   tomcat     "catalina.sh run"       2 minutes ago    Up 2 minutes    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp    strange_cori
2098b18efce8   jenkins/jenkins   "/usr/bin/tini -- /u..." 24 hours ago    Exited (143) 23 hours ago                    ecstatic_keldysh
a92a9feb0d42   jenkins/jenkins   "/usr/bin/tini -- /u..." 24 hours ago    Created                                           fervent_germain
3eeaf34205be   nginx      "/docker-entrypoint..." 24 hours ago    Exited (0) 23 hours ago                      jovial_keldysh

root@ip-172-31-4-171:~# docker exec -it 299ecd39c568 /bin/bash
root@299ecd39c568:/usr/local/tomcat# ls
bin      conf      filtered-KEYS  LICENSE  native-jni-lib  README.md  RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUTING.md  lib          NOTICE  RELEASE-NOTES  webapps
root@299ecd39c568:/usr/local/tomcat# cd webapps
root@299ecd39c568:/usr/local/tomcat/webapps# ll
total 12
drwxr-xr-x 2 root root 4096 Aug 21 19:01 ./
drwxr-xr-x 1 root root 4096 Aug 21 19:01 ../
root@299ecd39c568:/usr/local/tomcat/webapps#
```



Enter Tomcat Container

- `docker exec -it <container_id> /bin/bash`

Example:

- `docker exec -it 354d9e41fed /bin/bash`

What happened:

- Opens terminal **inside the container**.

Result: Now you are working inside Tomcat.

Go to Webapps Folder

- `cd /usr/local/tomcat/webapps`

Is

What happened:

- Goes to Tomcat deployment folder.

Is shows files inside.

Update Package List

- `apt update`

- Updates package information.

Result: System is ready to install software.

Install Wget Tool

- `apt install wget -y`

- Installs wget (used to download files).
- `-y` → Auto yes

```
root@299ecd39c568:/usr/local/tomcat/webapps# apt update
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
root@299ecd39c568:/usr/local/tomcat/webapps# apt install wget -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

Download Sample App

wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war

What happened:

- Downloads sample.war file.
- WAR = Java web application.

Result: sample.war saved in webapps folder

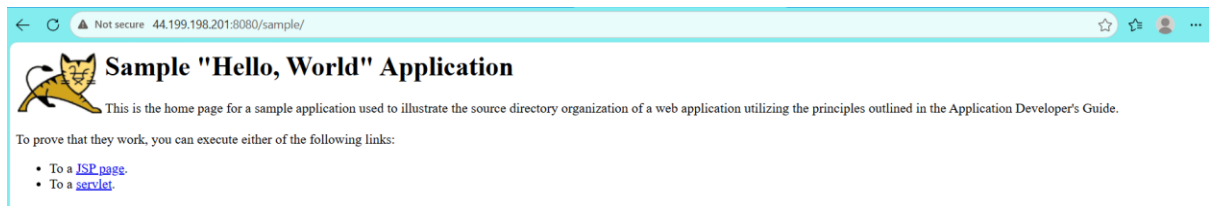
```
root@299ecd39c568:/usr/local/tomcat/webapps# wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
--2026-08-25 14:29:58-- https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
length: 4606 (4.5K)
Saving to: 'sample.war'

sample.war 100% [=====] 4.50K --.-KB/s in 0s
```

access Sample App in Browser

http://44.199.198.201/sample/

- Browser shows Hello World Sample App



2. CREATE VOLUME AND DEPLOY TOMCAT CONTAINER ON PORT 8081.

Create a Docker Volume

```
> docker volume create deploy
```

What happened:

- Creates a **named volume** called deploy.
- Docker will store data for this volume permanently

List Docker Volumes

```
> docker volume ls
```

Shows all Docker volumes.

Go to Volume Location

```
> cd /var/lib/docker/volumes
> ls
```

- Opens Docker's internal storage folder.
- Shows folders of all volumes

Enter Deploy Volume Folder

```
> cd deploy/_data
```

Download Sample App in Volume

```
wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
```

What happened:

- Downloads sample.war directly into the volume.

```

root@ip-172-31-4-171:~# docker volume create depoly
depol
root@ip-172-31-4-171:~# docker volume ls
DRIVER      VOLUME NAME
local       2ca32cdad9d73372308ee58a3203bcdcdcd1c1f2182fdc0b51e5900d46ac8efd
local       d7a31d78ac48208f4e06b802aae2e254242bf6aa2a59dcd666d3756d8408f5c6
local       depoly
local       e837ef8f8dff18a48663b6ebb671ceca58172be40ae70b0b15f1c1936c104482
local       ef33ca8047af36423c71c2dca58bf6c805b509df9514f54edc44722d8733980d
root@ip-172-31-4-171:~# cd /var/lib/docker/volumes
root@ip-172-31-4-171:/var/lib/docker/volumes# ls
2ca32cdad9d73372308ee58a3203bcdcdcd1c1f2182fdc0b51e5900d46ac8efd  depoly
backingfsBloCkDev                                                    e837ef8f8dff18a48663b6ebb671ceca58172be40ae70b0b15f1c1936c104482  metadata.db
d7a31d78ac48208f4e06b802aae2e254242bf6aa2a59dcd666d3756d8408f5c6  ef33ca8047af36423c71c2dca58bf6c805b509df9514f54edc44722d8733980d
root@ip-172-31-4-171:/var/lib/docker/volumes# cd /deploy
-bash: cd: /deploy: No such file or directory
root@ip-172-31-4-171:/var/lib/docker/volumes# cd d
d7a31d78ac48208f4e06b802aae2e254242bf6aa2a59dcd666d3756d8408f5c6/ depoly/
root@ip-172-31-4-171:/var/lib/docker/volumes# cd depoly/
root@ip-172-31-4-171:/var/lib/docker/volumes/depoly# ls
_data
root@ip-172-31-4-171:/var/lib/docker/volumes/depoly# cd _data/
root@ip-172-31-4-171:/var/lib/docker/volumes/depoly/_data# wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
--2026-08-25 14:59:57-- https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)[151.101.2.132]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4606 (4.5K)
Saving to: 'sample.war'

root@ip-172-31-4-171:/var/lib/docker/volumes/depoly/_data# ls
sample.war
root@ip-172-31-4-171:/var/lib/docker/volumes/depoly/_data#

```

Run Tomcat with Volume Mapping

- docker container run -itd -p 8083:8080 -v /var/lib/docker/volumes/deploy/_data:/usr/local/tomcat/webapps tomcat

What happened:

Part	Meaning
-itd	Run in background
-p 8083:8080	Host port → Container port
-v ...	Mount volume to Tomcat webapps
tomcat	Image

```

ubuntu@ip-172-31-4-171:~$ sudo su -
root@ip-172-31-4-171:~# docker container run -itd -p 8083:8080 -v /var/lib/docker/volumes/deploy/_data:/usr/local/tomcat/webapps tomcat
02c854ff16195f24f559e004566ff2628455383797de27c3bc25895b5852c68c
root@ip-172-31-4-171:~#

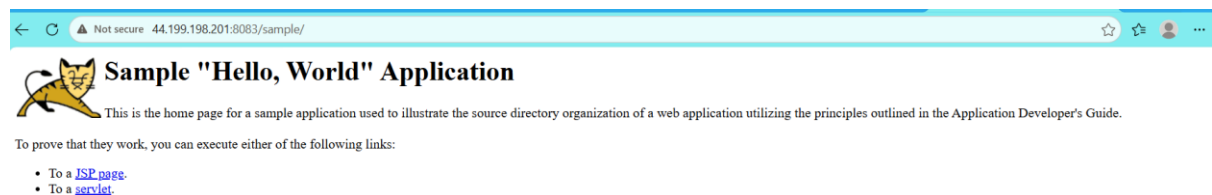
```

Open App in Browser

<http://44.199.198.201:8083/sample>

What happened:

- Browser loads app from volume.
- App works without copying files inside container.



3. LIMIT THE NGINX CONTAINER TO 500 MB.

Step 1: Run Nginx Container with Memory Limit

➤ `docker container run -itd -p 82:80 --memory 500m nginx`

Part	Meaning
<code>docker container run</code>	Create + start container
<code>-i</code>	Interactive
<code>-t</code>	Terminal
<code>-d</code>	Background mode
<code>-p 82:80</code>	Host port 82 → Container port 80
<code>--memory 500m</code>	Limit RAM to 500 MB
<code>nginx</code>	Image name

Monitor Resource Usage

➤ `docker stats`

- Shows live CPU & Memory usage.

Key field:

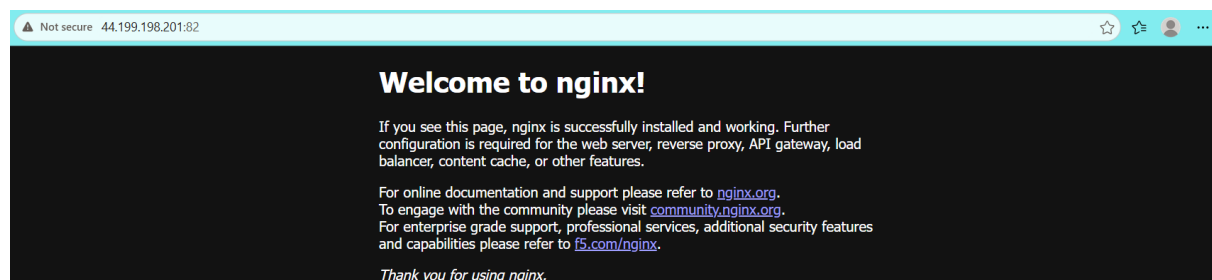
MEM USAGE / LIMIT → 7.77MiB / 500MiB

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
3121145b578c	sharp_buck	0.00%	14.67MiB / 500MiB	2.93%	726B / 126B	13.4MB / 12.3kB	3
02c854ff1619	friendly_heisenberg	0.05%	92.33MiB / 7.593GiB	1.19%	3.33kB / 4.28kB	274kB / 508kB	35
aafb383deaae	agitated_bouman	0.05%	74.47MiB / 7.593GiB	0.96%	5.66kB / 6.41kB	0B / 303kB	35
ef33ec533b83	gracious_turing	0.05%	78.73MiB / 7.593GiB	1.01%	7.68kB / 8.32kB	86kB / 479kB	35
299ecd39c568	strange_cori	0.20%	145MiB / 7.593GiB	1.86%	13.4kB / 123kB	4.15MB / 1.05MB	35

Open Nginx in Browser

`http://44.199.198.201:82`

Browser accessed container via port 82



4. CREATE A SAMPLE DOCKER FILE USING BELOW INSTRUCTIONS.

BASE MODULE AS AMAZONLINUX:LATEST

MAINTAINER YOUR NAME

INSTALL NGINX

COPY ONE INDEX.HTML FILE TO IMAGE

EXPOSE ON PORT 80

COMMAND TO START THE NGINX CONTAINER

Create Project Folder

- mkdir nginx-task
- cd nginx-task

Create Custom Web Page (index.html)

vi index.html

File Created: index.html

```
<html>
<head>
  <title>My Docker Nginx</title>
</head>
<body>
  <h1>Welcome to Hari Docker Nginx Server</h1>
  <p>Running on Amazon Linux</p>
</body>
</html>
```

Create Dockerfile

vi Dockerfile

File Created: Dockerfile

1. Base image

FROM amazonlinux:latest

2. Maintainer info

LABEL maintainer="mujju"

3. Install nginx

RUN yum update -y && \

yum install -y nginx && \

```
yum clean all
```

4. Copy website file

```
COPY index.html /usr/share/nginx/html/index.html
```

5. Open port 80

```
EXPOSE 80
```

6. Start nginx

```
CMD ["nginx", "-g", "daemon off;"]
```

```
root@ip-172-31-4-171:~# mkdir nginx-task
root@ip-172-31-4-171:~# cd nginx-task/
root@ip-172-31-4-171:~/nginx-task# vi index.html
root@ip-172-31-4-171:~/nginx-task# cat index.html
<html>
<head>
  <title>My Docker Nginx</title>
</head>
<body>
  <h1>Welcome to Hari Docker Nginx Server</h1>
  <p>Running on Amazon Linux</p>
</body>
</html>

root@ip-172-31-4-171:~/nginx-task# vi dockerfile
root@ip-172-31-4-171:~/nginx-task#
```

```
# 1. Base image
FROM amazonlinux:latest

# 2. Maintainer info
LABEL maintainer='Hari'

# 3. Install nginx
RUN yum update -y && \
    yum install -y nginx && \
    yum clean all

# 4. Copy website file
COPY index.html /usr/share/nginx/html/index.html

# 5. Open port 80
EXPOSE 80

# 6. Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

Build Docker Image

➤ `docker build --no-cache -t amazon-nginx .`

What happened:

- Builds image using Dockerfile.
- `-t amazon-nginx` → Image name.
- `.` → Current folder.

```

root@ip-172-31-4-171:~/nginx-task# vi dockerfile
root@ip-172-31-4-171:~/nginx-task# ls
dockerfile  index.html
root@ip-172-31-4-171:~/nginx-task# docker build --no-cache -t amazon-nginx .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

unknown flag: --no-cache

Usage:  docker build [OPTIONS] PATH | URL | -

Run 'docker build --help' for more information
root@ip-172-31-4-171:~/nginx-task# docker build --no-cache -t amazon-nginx .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/6 : FROM amazonlinux:latest
latest: Pulling from library/amazonlinux
f206cad06171: Pulling fs layer
dfb37c4bb4e4: Download complete
1b0dff64fdeb: Download complete
f206cad06171: Download complete
f206cad06171: Pull complete

```

Run Container from Custom Image

- `docker run -d -p 9090:80 --name amazon-nginx-app amazon-nginx`

What happened:

Part	Meaning
-d	Run in background
-p 9090:80	Host 9090 → Container 80
--name	Container name
amazon-nginx	Image name

```

root@ip-172-31-4-171:~/nginx-task# docker run -d -p 9090:80 --name amazon-nginx-app amazon-nginx
e3c8b5c4c8d3b9a4c154786a486c739043dd45179f9d8e7a5cd701ade544bd6d
root@ip-172-31-4-171:~/nginx-task# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
e3c8b5c4c8d3  amazon-nginx  "nginx -g 'daemon of..." 7 seconds ago  Up 6 seconds  0.0.0.0:9090->80/tcp, [::]:9090->80/tcp  amazon-nginx-app
3121145b578c  nginx         "/docker-entrypoint..." 27 minutes ago Up 27 minutes  0.0.0.0:82->80/tcp, [::]:82->80/tcp      sharp_buck
02c854ff1619  tomcat        "catalina.sh run"        32 minutes ago Up 32 minutes  0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp  friendly_heisenberg
earfb383deaae  tomcat        "catalina.sh run"        43 minutes ago Up 43 minutes  0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp  agitated_bouman
299ecd39c568  tomcat        "catalina.sh run"        2 hours ago   Up About an hour  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  strange_cori
root@ip-172-31-4-171:~/nginx-task#

```

Open in Browser

<http://44.199.198.201:9090>

What happened:

- Opens your custom website.



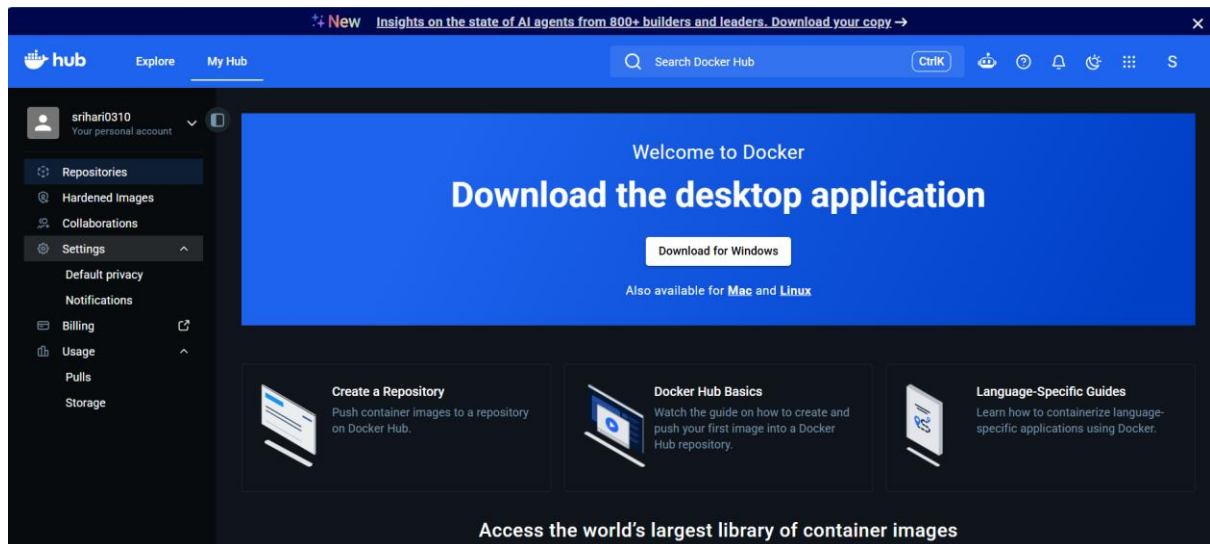
5. PUSH IMAGE TO DOCKER HUB

Open Docker Hub Dashboard

considered site:

<https://hub.docker.com>

- logged into Docker Hub.
- Opened **Repositories** section

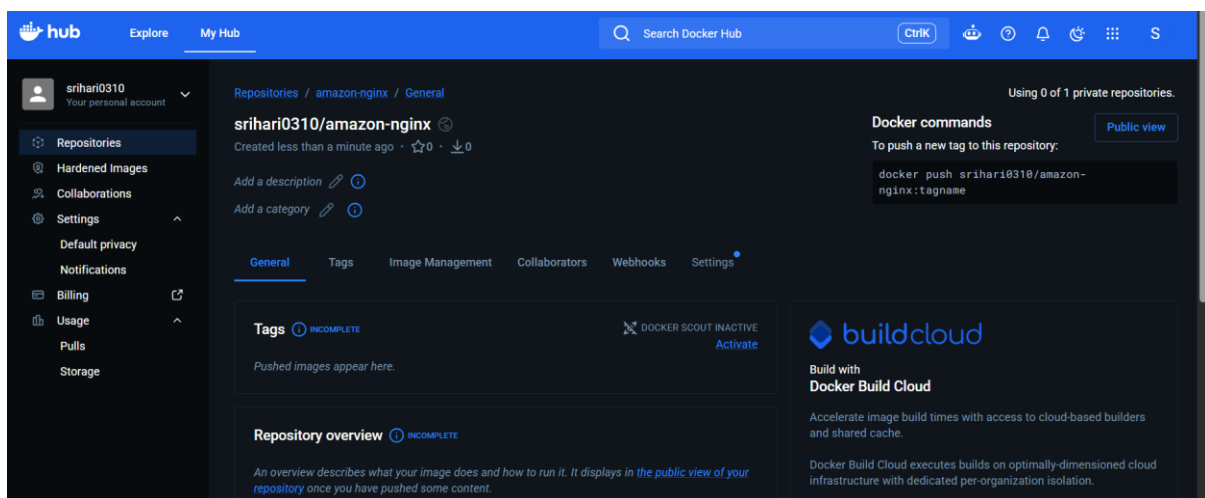
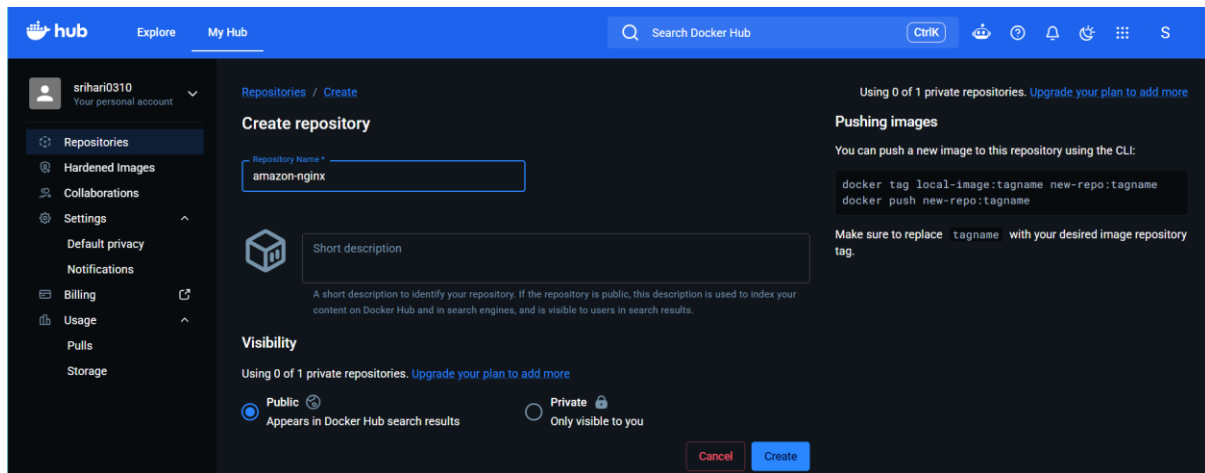


Create a New Repository

On Docker Hub website:

1. Click **Repositories**
 2. Click **Create Repository**
 3. Enter name:
 4. amazon-nginx
 5. Select **Public**
 6. Click **Create**
- Created an online storage place for your image.

Result: Repository srihari0310/amazon-nginx created.



Login to Docker Hub from Server

- `docker login -u srihari0310`
- Connected your EC2 server to Docker Hub.
- Entered username & password

```
root@ip-172-31-4-171:~# docker login -u srihari0310
Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings
Password:
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
root@ip-172-31-4-171:~#
```

Check Local Images

- `docker images`
- Listed images on your server.

Result: saw amazon-nginx image.

Tag Image for Docker Hub

- `docker tag amazon-nginx srihari0310/amazon-nginx`
- Renamed image to match Docker Hub format.

Format:

username/repository:tag

Here:

Srihari0310/amazon-nginx:latest

```
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	0
amazonlinux:latest	181f98c48832	227MB	59.3MB	0
jenkins/jenkins:latest	f296d729ad09	707MB	243MB	0
nginx:latest	0d4374c710a9	241MB	66.2MB	0
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	0
tomcat:latest	b4237a8551b3	589MB	158MB	0
ubuntu:latest	2260313b31c8	160MB	45.3MB	0

```
root@ip-172-31-4-171:~# docker tag amazon-nginx srihari0310/amazon-nginx
root@ip-172-31-4-171:~# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e3c8b5c4c8d3	amazon-nginx	"nginx -g 'daemon of..."	19 minutes ago	Up 19 minutes	0.0.0.0:9090->80/tcp, [::]:9090->80/tcp	amazon-nginx-app
3121145b578c	nginx	"/docker-entrypoint..."	46 minutes ago	Up 46 minutes	0.0.0.0:82->80/tcp, [::]:82->80/tcp	sharp_buck
02c854ff1619	tomcat	"catalina.sh run"	51 minutes ago	Up 51 minutes	0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp	friendly_heisenberg
earb383deaae	tomcat	"catalina.sh run"	About an hour ago	Up About an hour	0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp	agitated_bouman
299ecd39c568	tomcat	"catalina.sh run"	2 hours ago	Up 2 hours	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	strange_cori

```
root@ip-172-31-4-171:~#
```

push Image to Docker Hub

➤ docker push srihari0310/amazon-nginx

- Uploaded image to Docker Hub.

```
root@ip-172-31-4-171:~# docker push srihari0310/amazon-nginx
Using default tag: latest
The push refers to repository [docker.io/srihari0310/amazon-nginx]
ecf0fe886c5d: Pushed
f206cad06171: Mounted from library/amazonlinux
18636a858044: Pushed
latest: digest: sha256:55f7b41c58108699ffe36ac7f561844c8dc8d1a03a43c8f51243cf89c18a72d size: 971
root@ip-172-31-4-171:~#
```

Verify on Docker Hub Website

The screenshot shows the Docker Hub interface for the repository **srihari0310/amazon-nginx**. The page is titled "Repositories / amazon-nginx / General". It shows the repository was last pushed 1 minute ago and has a size of 74.2 MB. The "Tags" section indicates that the repository contains 0 tags. The "Docker commands" section provides the command to push a new tag: `docker push srihari0310/amazon-nginx:tagname`. The "Tags" table is empty, showing no tags are currently listed. The "General" tab is selected, and other tabs like "Tags", "Image Management", "Collaborators", "Webhooks", and "Settings" are visible. The "Docker Scout" status is shown as "INACTIVE" with an "Activate" button. The "Build with Docker Build Cloud" section is also visible, promoting cloud-based builders and shared cache.

6. PUSH IMAGE TO AWS ECR

- aws configure

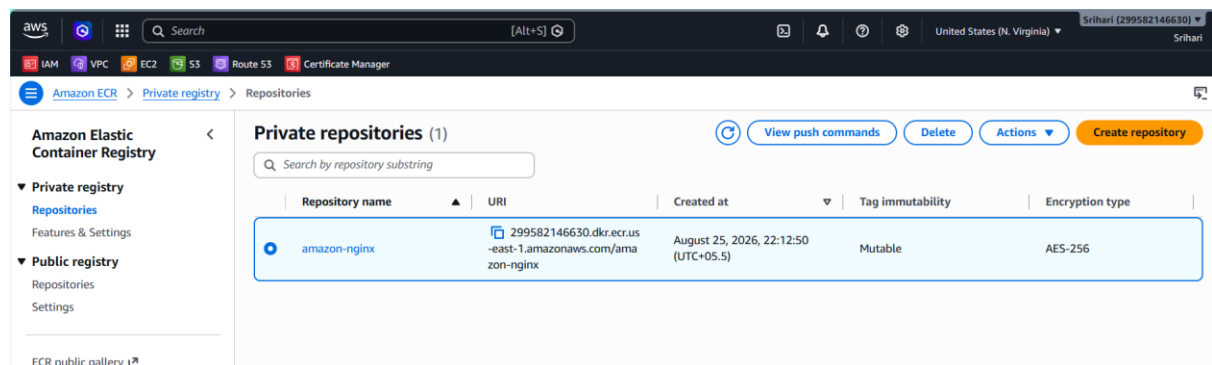
```
root@ip-172-31-4-171:~# aws configure
AWS Access Key ID [None]: AKIAULQD4ZBDMRP77UJG
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]: json
root@ip-172-31-4-171:~#
```

Create ECR Repository

- aws ecr create-repository --repository-name amazon-nginx --region eu-north-1

```
root@ip-172-31-4-171:~# aws ecr create-repository --repository-name amazon-nginx --region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:299582146630:repository/amazon-nginx",
    "registryId": "299582146630",
    "repositoryName": "amazon-nginx",
    "repositoryUri": "299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx",
    "createdAt": "2026-08-25T16:42:50.886000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

- verified repository is created in aws ecr



Login to ECR --- Replace ACCOUNT_ID with your AWS account number

- aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 299582146630.dkr.ecr.us-east-1.amazonaws.com

```
root@ip-172-31-4-171:~# aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 299582146630.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-4-171:~#
```

Tag Your Image

- docker tag amazon-nginx:latest 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest

```
root@ip-172-31-4-171:~# docker tag amazon-nginx:latest 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest
root@ip-172-31-4-171:~# docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	U
299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	U
amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	U
amazonlinux:latest	181f98c48832	227MB	59.3MB	
jenkins/jenkins:latest	f296d729ad09	707MB	24.3MB	U
nginx:latest	0d4374c710a9	241MB	66.2MB	U
sonarqube:latest	9026624a61cd	2.53GB	1.09GB	
srihari0310/amazon-nginx:latest	55f7b41c5810	318MB	77.8MB	U
tomcat:latest	b4237a8551b3	589MB	158MB	
ubuntu:latest	2260313b31c8	160MB	45.3MB	U

Push Image

➤ `docker push 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest`

```
root@ip-172-31-4-171:~# docker push 299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest
The push refers to repository [299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx]
18636a858044: Pushed
eef0fe886c5d: Pushed
f206cad06171: Pushed
latest: digest: sha256:55f7b41c58108699ffe36ac7f561844c8dcb8d1a03a43c8f51243cf89c18a72d size: 971
root@ip-172-31-4-171:~#
```

Verified the pushed image by going to aws ecr and image was pushed

The screenshot shows the AWS Management Console for Amazon Elastic Container Registry (ECR). The breadcrumb navigation at the top reads: **Amazon ECR** > **Private registry** > **Repositories** > **amazon-nginx** > **sha256:55f7b41c58108699ffe36ac7f561844c8dcb8d1a03a43c8f51243cf89c18a72d**. The left-hand navigation pane is titled "Amazon Elastic Container Registry" and includes sections for "Private registry" (Repositories, amazon-nginx, Features & Settings) and "Public registry" (Repositories, Settings). Below these are links for "ECR public gallery", "Amazon ECS", "Amazon EKS", "Getting started", and "Documentation". The main content area is titled "Image" and contains a section for "Details". This section lists the "Image tags" as "latest", the "URI" as "299582146630.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:latest", and the "Digest" as "sha256:55f7b41c58108699ffe36ac7f561844c8dcb8d1a03a43c8f51243cf89c18a72d". A "General information" table follows, with columns for "Artifact type" (Image), "Repository" (amazon-nginx), "Pushed at" (August 25, 2026, 22:43:39 (UTC+05.5)), "Last recorded pull time" (-), "Size (MB)" (77.82), and "Image status" (Active). At the bottom of the details section is a "Scanning and vulnerabilities" section with a "Scan" button. An "Archive" button is located in the top right corner of the details section.

General information		
Artifact type	Repository	Pushed at
Image	amazon-nginx	August 25, 2026, 22:43:39 (UTC+05.5)
Last recorded pull time	Size (MB)	Image status
-	77.82	Active